Relatório — StateFlow

Navegação e gerenciamento de estado no React Native

Aluno(a): [Seu nome] Disciplina: [Nome da disciplina] Instituição: [Nome]

1. Introdução

Aplicações móveis precisam apresentar telas, responder às interações e compartilhar informações entre componentes. No React, estado representa dados que podem mudar durante a execução e cuja alteração pode provocar nova renderização da interface. Este trabalho demonstra navegação e gerenciamento de estado em React Native.

2. Estado e hooks

Estado local pertence a um componente, como o texto digitado ou um contador. **Estado compartilhado/global** é consumido por diferentes partes da aplicação, como o usuário autenticado. **useState** cria e atualiza estado; **useEffect** executa efeitos colaterais conforme as dependências; **useContext** lê um valor fornecido por um Context Provider sem repassar props por muitos níveis.

3. Navegação

React Navigation é uma biblioteca de navegação para React Native. Stack Navigator organiza telas em uma pilha: navegar normalmente adiciona uma rota, voltar remove a rota superior e substituir troca a rota atual. Tab Navigator oferece destinos paralelos, como Início e Perfil. Neste app, o Stack apresenta Login quando não existe usuário e mostra a área com abas quando o contexto contém um usuário.

4. Context API

AuthProvider armazena o objeto user e oferece user, signIn e signOut. Login chama signIn; Dashboard e Perfil leem os mesmos dados por useContext; Perfil chama signOut. A Context API é suficiente para este projeto por haver poucas telas e um estado compartilhado simples.

5. Comparação de soluções

Solução	Quando usar	Vantagens	Limitações
Context API	Estado simples compartilhado	Nativa, direta	Atenção a re-renderizações
Redux Toolkit	Estado complexo e fluxos centralizados	DevTools e padrões maduros	Mais conceitos e estrutura
Zustand	Store global enxuta	API simples, seletores	Dependência e padrões a definir
Recoil	Modelo por átomos e seletores	Estado derivado granular	Projeto arquivado; cautela em novos projetos

6. Escolha arquitetural e desempenho

A Context API foi escolhida porque o aplicativo tem poucas telas e um estado global central. Redux Toolkit seria mais adequado se existissem vários domínios, fluxos complexos e necessidade forte de depuração. Zustand é uma alternativa enxuta se o estado crescer. Recoil não foi escolhido porque o repositório foi arquivado. Para reduzir renderizações desnecessárias, o código usa useMemo para memorizar o valor do Provider e useCallback para estabilizar funções. Em sistemas maiores, separar contextos e componentes ajuda; a memoização deve ser guiada por medição.

7. Conclusão

O aplicativo demonstra navegação condicional com Stack Navigator, abas com Tab Navigator e compartilhamento do usuário por Context API. A escolha da ferramenta depende da complexidade, frequência das atualizações, necessidade de depuração e manutenção. O login deste exemplo é apenas demonstrativo e não substitui autenticação real.

Referências

React. Built-in React Hooks. https://react.dev/reference/react

React Navigation. Documentation. https://reactnavigation.org/docs/getting-started

Redux Toolkit. https://redux-toolkit.js.org/

Zustand. https://zustand.docs.pmnd.rs/

Recoil repository. https://github.com/facebookexperimental/Recoil